

VIP скрипта:

Трансформери и велики језички модели

Afshine AMIDI и Shervine AMIDI

Превео: Ненад Мијатовић

24. маја 2026.

Ова VIP скрипта представља преглед књиге “Super Study Guide: Transformers & Large Language Models”, која на 250 страница и кроз више од ~600 илустрација детаљно обрађује концепте приказане овде. За више детаља, посетите страницу <https://superstudy.guide>.

1 Основе

1.1 Токени

**Дефиниција** – Токен је недељива јединица текста, као што је реч, део речи или слово, и представља део унапред дефинисаног речника.

*Напомена: Непознати токен [UNK] представља непознате делове текста, док се токен за попуњавање [PAD] користи да попуни празне позиције како би улазни низови имали исту дужину.*

**Токенизатор** – Токенизатор  $T$  дели дати текст на токене на произвољном нивоу грануларности.

плишани меда је баааш сладак  $\rightarrow$   $T$   $\rightarrow$  [плишани меда] [је] [UNK] [сладак] [PAD] ... [PAD]

Следећа табела приказује главне типове токенизатора:

| Тип           | Предности                                                                                        | Мане                                                                                                                        | Пример                                |
|---------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| Реч           | <ul style="list-style-type: none"><li>Разумљивост</li><li>Кратки низови</li></ul>                | <ul style="list-style-type: none"><li>Велики речник</li><li>Варијације речи нису обрађене</li></ul>                         | [плишани]<br>[медвед]                 |
| Део речи      | <ul style="list-style-type: none"><li>Користи корен речи</li><li>Интуитивна уграђивања</li></ul> | <ul style="list-style-type: none"><li>Повећава дужину низова</li><li>Токенизација је сложенија</li></ul>                    | [плиш ##ани]<br>[мед ##а]             |
| Слово<br>Бајт | <ul style="list-style-type: none"><li>Нема проблема ван речника</li><li>Мали речник</li></ul>    | <ul style="list-style-type: none"><li>Много дужи низови</li><li>Обрасци су тешко разумљиви јер су прениског нивоа</li></ul> | [п][л][и][ш][а][н][и]<br>[м][е][д][а] |

*Напомена: Кодирање парова бајтова (Byte-Pair Encoding, BPE) и Unigram су често коришћени токенизатори на нивоу делова речи.*

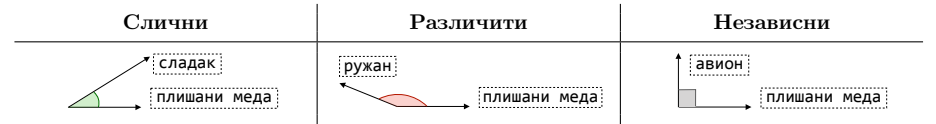
1.2 Уграђивања

**Дефиниција** – Уграђивање је нумеричка репрезентација елемента (нпр. токена или реченице) и описано је вектором  $x \in \mathbb{R}^n$ .

**Сличност** – Косинусна сличност између два токена  $t_1, t_2$  израчунава се као:

$$\text{similarity}(t_1, t_2) = \frac{t_1 \cdot t_2}{\|t_1\| \|t_2\|} = \cos(\theta) \in [-1, 1]$$

Угао  $\theta$  карактерише сличност између два токена:

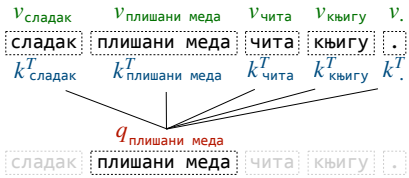


*Напомена: Approximate Nearest Neighbors (ANN) и Locality Sensitive Hashing (LSH) су методи који ефикасно апроксимирају операцију сличности над великим базама података.*

2 Трансформери

2.1 Пажња

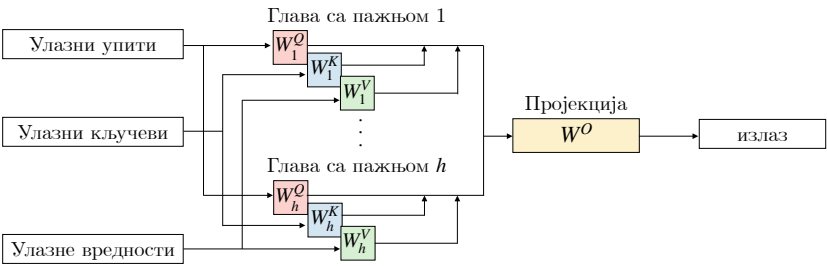
**Формула** – За задати упит  $q$  (query) желимо да одредимо на који кључ  $k$  (key) упит треба да обрати “пажњу” у односу на придружену вредност  $v$  (value).



Пажња се ефикасно израчунава коришћењем матрица  $Q, K, V$ , које редом садрже упите  $q$ , кључеве  $k$  и вредности  $v$ , заједно са димензијом кључева  $d_k$ :

$$\text{attention} = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V$$

**Пажња са више глава** – Слој пажње са више глава (*Multi-Head Attention*, МНА) израчунава пажњу кроз више глава, а затим пројектује резултат у излазни простор.

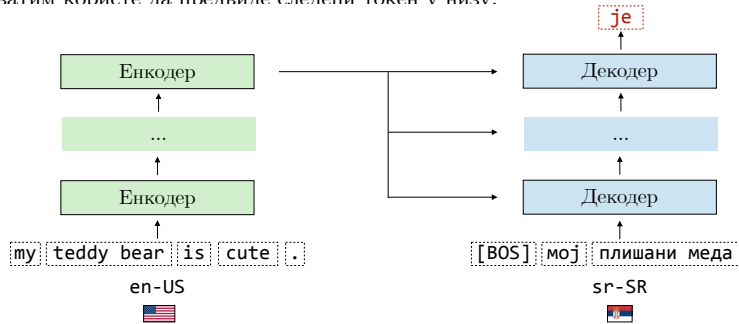


Састоји се од  $h$  глава пажње, као и матрица  $W^Q, W^K, W^V$  које пројектују улаз како би се добили упити  $Q$ , кључеви  $K$  и вредности  $V$ . Излазна пројекција се извршава матрицом  $W^O$ .

*Напомена: Grouped-Query Attention (GQA) и Multi-Query Attention (MQA) су варијанте MHA које смањују рачунско оптерећење дељењем кључева и вредности између глава пажње.*

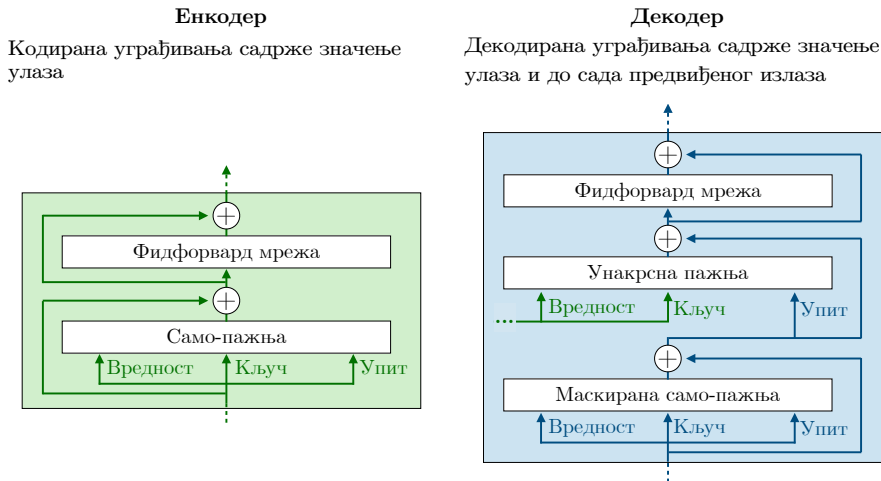
## 2.2 Архитектура

□ **Преглед** – Трансформер је значајан модел који се ослања на механизам самопажње и састоји се од енкодера и декодера. Енкодери рачунају смислена уграђивања улаза, која декодери затим користе да прелвине следећи токен у низу.



*Напомена: Иако је Трансформер првобитно предложен као модел за преводјење, сада се широко користи у многим другим применама.*

□ **Компоненте** – Енкодер и декодер су две основне компоненте Трансформера и имају различите улоге:

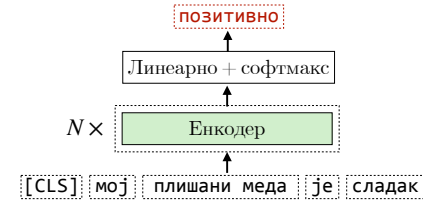


□ **Уграђивања позиција** – Уграђивања позиција показују где се токен налази у реченици и имају исту димензију као уграђивања токена. Могу бити произвољно дефинисана или научена из података.

*Напомена: Rotary Position Embeddings (RoPE) су популарна и ефикасна варијанта која ротира векторе упита и кључева како би укључила информације о релативној позицији.*

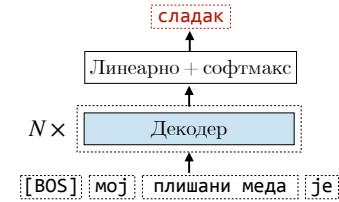
## 2.3 Варијанте

□ **Самостални енкодер** – BERT (*Bidirectional Encoder Representations from Transformers*) је модел заснован на Трансформеру и састоји се од стека енкодера. Као улаз прима текст, а као излаз враћа смислена уграђивања која се касније могу користити у низводним класификационим задацима.



[CLS] токен се додаје на почетак низа како би ухватио значење реченице. Његово кодирано уграђивање се често користи у низводним задацима, као што је детекција сенитета.

□ **Самостални декодер** – Генеративни претходно истренирани трансформер (*Generative Pre-trained Transformer*, GPT) је ауторегресивни модел заснован на Трансформеру и састоји се од стека декодера. За разлику од BERT-а и његових изведеница, GPT све проблеме третира као проблеме типа текст-у-текст.



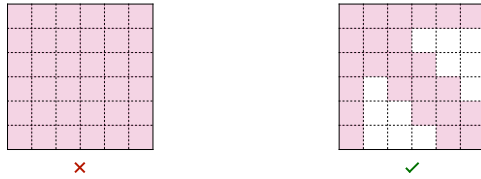
Већина данашњих најнапреднијих LLM-ова ослања се на архитектуру са самосталним декодером, као што су GPT серија, LLaMA, Mistral, Gemma, DeepSeek итд.

*Напомена: Енкодер-декодер модели, као што је T5, такође су ауторегресивни и деле многе карактеристике са моделима који користе само декодер.*

## 2.4 Оптимизација

□ **Апроксимација пажње** – Рачунање пажње је сложености  $\mathcal{O}(n^2)$ , што може бити скупо како дужина низа  $n$  расте. Постоје два главна метода за апроксимацију рачунања:

- *Разређеност (Sparsity)*: самопажња се не рачуна кроз цео низ, већ само између релевантнијих токена.



- **Низак ранг (Low-rank):** формула пажње се поједностављује као производ матрица ниског ранга, чиме се смањује рачунско оптерећење.

□ **Брза пажња** – Брза пажња (*Flash attention*) је тачан метод који оптимизује израчунавање пажње вештим коришћењем GPU хардвера, тако што за матричне операције користи брзу статичку меморију са директним приступом (*Static Random-Access Memory*, SRAM), пре уписивања резултата у спорију меморију великог протока (*High Bandwidth Memory*, HBM).

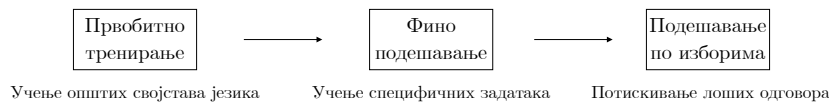
*Напомена: У пракси, овај метод смањује употребу меморије и убрзава израчунавања.*

## 3 Велики језички модели

### 3.1 Преглед

□ **Дефиниција** – Велики језички модел (*Large Language Model*, LLM) је модел заснован на Трансформеру са снажним способностима обраде природног језика (NLP). Назива се “великим” зато што обично садржи милијарде параметара.

□ **Животни циклус** – LLM се обично тренира у три корака: претходно тренирање, фина подешавање и подешавање на основу склоности.

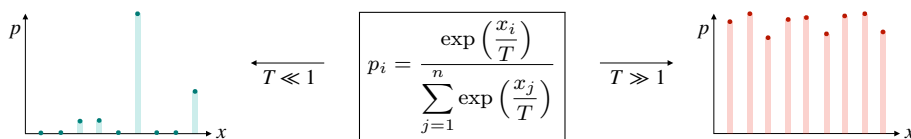


Фино подешавање и подешавање на основу склоности су приступи након тренирања чији је циљ да *ускладе модел* за извршавање одређених задатака.

### 3.2 Промптовање

□ **Дужина контекста** – Дужина контекста модела је максималан број токена који може да стане у један улаз. Обично се креће од десетина хиљада до милиона токена.

□ **Узорковање при декодирању** – Предвиђања токена се узоркују из предвиђене расподеле вероватноћа  $p_i$ , којом управља хиперпараметар температуре  $T$ .



*Напомена: Високе вредности температуре воде ка креативнијим излазима, док ниске вредности воде ка детерминистичкијим резултатима.*

□ **Ланац резоновања** – Ланац резоновања (*Chain-of-Thought*, CoT) је процес у ком модел разлаже сложен проблем на низ посредних корака. То помаже моделу да генерише тачан коначни одговор. Дрво мисли (*Tree of Thoughts*, ToT) напреднија је верзија CoT-а.

*Напомена: Самопостојаност (Self-consistency) је метод који агрегира одговоре дуж различитих CoT путања резоновања.*

### 3.3 Фино подешавање

□ **Надгледано фина подешавање** – Надгледано фина подешавање (*Supervised FineTuning*, SFT) је приступ након тренирања којим се понашање модела усклађује са крајњим задатком. Ослања се на висококвалитетне улазно-излазне парове усклађене са задатком.

*Напомена: Ако су SFT подаци у облику инструкција, онда се овај корак назива “подешавање инструкцијама”.*

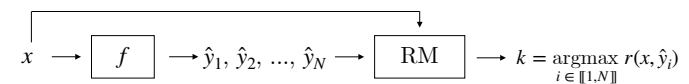
□ **Параметарски ефикасно фина подешавање** – Параметарски ефикасно фина подешавање (*Parameter-Efficient FineTuning*, PEFT) је категорија метода којима се SFT извршава ефикасно. Конкретно, адаптација ниског ранга (*Low-Rank Adaptation*, LoRA) апроксимира тежине  $W$  које се уче тако што фиксира  $W_0$  и уместо тога учи матрице ниског ранга  $A, B$ :

$$d \times k \text{ matrix } W \approx d \times k \text{ matrix } W_0 + d \times r \text{ matrix } B \times r \times k \text{ matrix } A$$

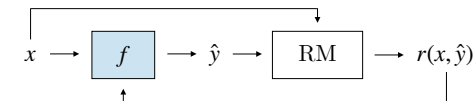
*Напомена: Друге PEFT технике укључују подешавање префикса и уметање адаптерског слоја.*

### 3.4 Подешавање на основу склоности

□ **Модел награђивања** – Модел награђивања (*Reward Model*, RM) је модел који предвиђа колико је неки излаз  $\hat{y}$  усклађен са жељеним понашањем за дати улаз  $x$ . Узорковање најбољи-од- $N$  (*Best-of-N*, BoN), познато и као узорковање са одбацивањем, користи модел награђивања да изабере најбољи одговор међу  $N$  генерисаних излаза.



□ **Појачано учење** – Појачано учење (*Reinforcement Learning*, RL) је приступ који користи RM и ажурира модел  $f$  на основу награда за његове генерисане излазе. Ако је RM заснован на људским склоностима, тај процес се назива појачано учење на основу људске повратне информације (*Reinforcement Learning from Human Feedback*, RLHF).

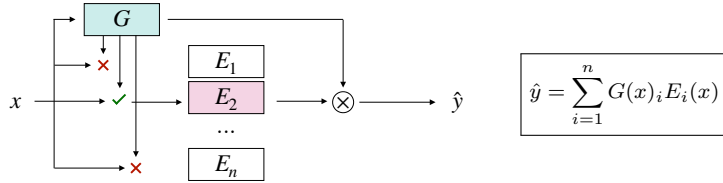


Проксимална оптимизација стратегије (*Proximal Policy Optimization*, PPO) је популаран RL алгоритам који подстиче веће награде, док модел држи близу основног модела како би се спречило хаковање награде.

*Напомена: Постоје и надгледани приступи, као што је Direct Preference Optimization (DPO), који комбинују RM и RL у један надгледани корак.*

### 3.5 Оптимизација

▢ **Мешавина експерата** – Мешавина експерата (*Mixture of Experts*, MoE) је модел који током извођења активира само део својих неурона. Заснива се на механизму распоређивања (*gate*)  $G$  и експертима  $E_1, \dots, E_n$ .



LLM-ови засновани на MoE архитектури користе овај механизам распоређивања у својим FFNN слојевима.

*Напомена: Тренирање LLM-ова заснованих на MoE архитектури познато је као веома захтевно, као што је истакнуто у раду о LLaMA моделу, чији су аутори одустали од ове архитектуре упркос њеној ефикасности током извођења.*

▢ **Дестилација** – Дестилација је процес у коме се мањи студентски модел  $S$  тренира на излазним предвиђањима већег учитељског модела  $T$ . Тренира се коришћењем губитка KL дивергенције:

$$\text{KL}(\hat{y}_T || \hat{y}_S) = \sum_i \hat{y}_T^{(i)} \log \left( \frac{\hat{y}_T^{(i)}}{\hat{y}_S^{(i)}} \right)$$

*Напомена: Ознаке за тренирање сматрају се “меким” ознакама јер представљају вероватноће класа.*

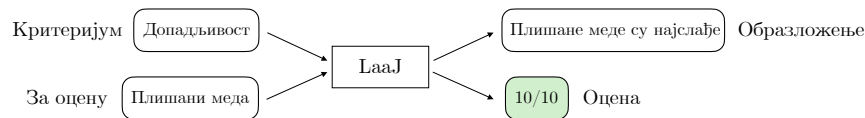
▢ **Квантизација** – Квантизација модела је категорија техника која смањује прецизност тежи-на модела, уз настојање да ограничи утицај на коначне перформансе модела. Као последица тога, смањује се меморијски отисак модела и убрзава његово извођење.

*Напомена: QLoRA је широко коришћена квантизована варијанта LoRA.*

## 4 Примене

### 4.1 LLM као судија

▢ **Дефиниција** – LLM као судија (*LLM-as-a-Judge*, LaaJ) је метод који користи LLM да оцењује дате излазе у складу са задатим критеријумима. Посебно је значајно то што може да генерише и образложење за додељену оцену, што доприноси интерпретабилности.



За разлику од метрика из периода пре LLM модела, као што је *Recall-Oriented Understudy for Gisting Evaluation* (ROUGE), LaaJ не захтева референтни текст, што га чини погодним за оцењивање било које врсте задатка. Посебно, LaaJ показује снажну корелацију са људским

оценама када се ослања на велики и моћан модел (нпр. GPT-4), јер су му за добар учинак неопходне способности резоновања.

*Напомена: LaaJ је користан за брзе рунде процене, али је важно пратити усклађеност између LaaJ излаза и људских процена како би се осигурало да не долази до одступања.*

▢ **Уобичајене пристрасности** – LaaJ модели могу испољавати следеће облике пристрасности:

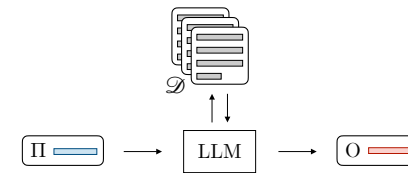
|         | Пристрасност према позицији                      | Пристрасност према опширности     | Пристрасност према себи                                      |
|---------|--------------------------------------------------|-----------------------------------|--------------------------------------------------------------|
| Проблем | Даје предност првој позицији у парним поређењима | Даје предност опширнијем садржају | Даје предност сопственим генерисаним излазима                |
| Решење  | Упросечити метрику на насумичним позицијама      | Додати казну на дужину излаза     | Користити судију изграђеног на основу другог основног модела |

Једно могуће решење за ове проблеме јесте додатно фино подешавање прилагођеног LaaJ модела, али то захтева много труда.

*Напомена: Наведена листа пристрасности није исцрпна.*

### 4.2 RAG

▢ **Дефиниција** – Генерисање потпомогнуто претраживањем (*Retrieval-Augmented Generation*, RAG) је метод који омогућава LLM моделу да приступи релевантном спољашњем знању како би одговорио на дато питање. Ово је посебно корисно када желимо да укључимо информације настале након датума до ког је LLM претходно трениран.



На основу базе знања  $\mathcal{D}$  и питања, модул за дохват (*Retriever*) проналази најрелевантније документе; затим се промпт проширује (*Augments*) релевантним информацијама пре генерисања (*Generating*) излаза.

*Напомена: Фаза дохвата се типично ослања на уграђивања из модела који садрже само енкодер.*

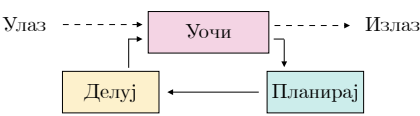
▢ **Хиперпараметри** – База знања  $\mathcal{D}$  иницијализује се дељењем докумената на сегменте величине  $n_c$  и њиховим уграђивањем у векторе димензије  $\mathbb{R}^d$ .



4.3 Агенти

□ **Дефиниција** – Агент је систем који самостално прати циљеве и извршава задатке у име корисника. За то може користити различите ланце позива LLM модела.

▣ **ReAct** – Разлог + акција (*Reason + Act*, ReAct) је оквир који омогућава коришћење више ланаца позива LLM модела ради извршавања сложених задатака:



Овај оквир се састоји од следећих корака:

- *Уочи (Observe)*: синтетизуј претходне акције и експлицитно наведи шта је тренутно познато.
- *Планирај (Plan)*: прецизирај које задатке треба обавити и које алате треба позвати.
- *Делуј (Act)*: изврши акцију путем API-ја или пронађи релевантне информације у бази знања.

*Напомена: Процњивање агентског система је захтевно. Ипак, може се спровести и на нивоу компоненти преко локалних улаза и излаза, као и на нивоу система преко ланаца позива.*

4.4 Модел резоновања

□ **Дефиниција** – Модел резоновања је модел који се ослања на трагове резоновања засноване на CoT-у како би решавао сложеније задатке из математике, програмирања и логике. Примери модела резоновања укључују OpenAI о серију, DeepSeek-R1 и Google Gemini Flash Thinking.

*Напомена: DeepSeek-R1 експлицитно исписује свој траг резоновања између <think> ознака.*

▣ **Скалирање** – Две врсте метода скалирања користе се за унапређење способности резоновања:

|                           | Опис                                                                                                   | Илустрација |
|---------------------------|--------------------------------------------------------------------------------------------------------|-------------|
| Скалирање током тренирања | Покренути RL дуже како би модел научио да производи трагове резоновања у CoT стилу пре давања одговора |             |
| Скалирање током тестирања | Дозволити моделу да дуже “размишља” пре одговора уз кључне речи за принуду буџета, као што је “Wait”   |             |